we now extract the predictive standard deviation like so:

```
predictive_stddev = predictive_distribution.stddev
```

Now, to draw the heat maps, we use the `imshow()` function from Matplotlib. We need to be careful with the shape of our predictions in `predictive_mean` and `predictive_stddev` here. Each of them is a tensor of length 10,000, so it will need to be reshaped into a square matrix before being passed to the `imshow()` function. This can be done as follows:

```
fig, ax = plt.subplots(1, 2)

ax[0].imshow(
    predictive_mean.detach().reshape(101, 101).transpose(-1, -2),
    origin="lower",
    extent=[-3, 3, -3, 3]
)  ❶

ax[1].imshow(
    predictive_stddev.detach().reshape(101, 101).transpose(-1, -2),
    origin="lower",
    extent=[-3, 3, -3, 3]
)  ❷
```

❶ The first heat map for the predictive mean.

❷ The second heat map for the predictive standard deviation.

This code produces the two heat maps in Figure 2.19.

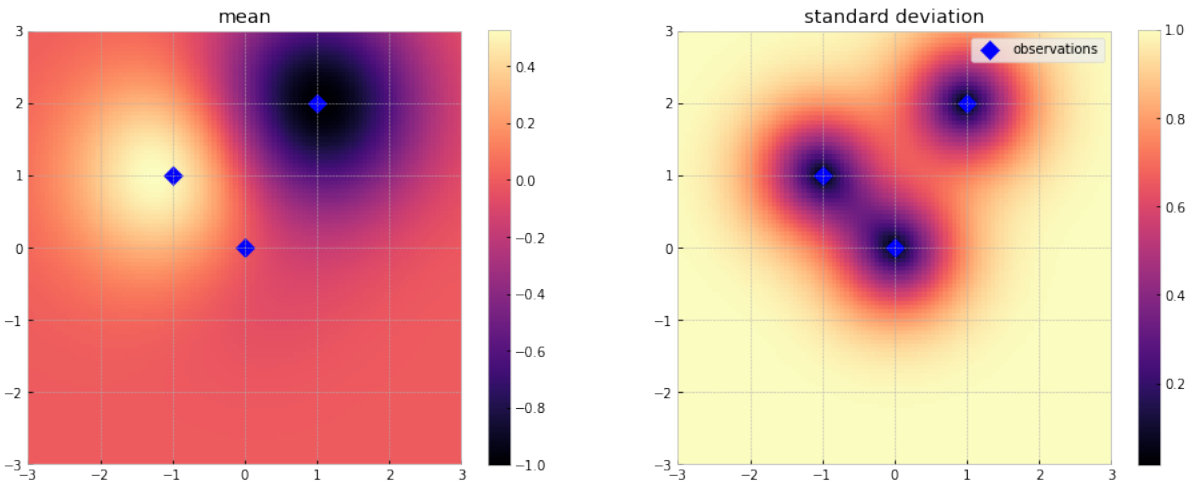


Figure 2.19 Predictions made by a two-dimensional GP. The mean function still agrees with the training data, and uncertainty once again vanishes in the regions surrounding these data points.

We see that what we have in the one-dimensional case extends to this example too.

- The left panel shows that our mean prediction agrees with our training data: the bright blob on the left corresponds to (-1, 1), which has a value of 0.5, while the dark blob on the right corresponds to (1, 2) having a value of -1 (our observation at (0, 0) has a value of 0 which is also the prior mean, so it is not as obvious to point out in the left panel).

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>